

```

# =====
# GENERATIVE AI LAB — COMPLETE IMPLEMENTATION (Q1–Q10)
# =====

import numpy as np
import matplotlib.pyplot as plt
import random, math
from collections import defaultdict, Counter
from sklearn.datasets import make_blobs
from sklearn.linear_model import LogisticRegression
from sklearn.mixture import GaussianMixture
from sklearn.metrics import accuracy_score, mean_squared_error
from sklearn.decomposition import PCA
from sklearn.neural_network import MLPClassifier
from scipy.optimize import linear_sum_assignment

print("\n===== Q1: Generative vs Discriminative =====\n")

# -----
# Dataset
# -----
X, y = make_blobs(n_samples=400, centers=2, random_state=42)

# Logistic Regression (Discriminative)
lr = LogisticRegression()
lr.fit(X, y)
lr_pred = lr.predict(X)
lr_acc = accuracy_score(y, lr_pred)

# Gaussian Mixture Model (Generative)
gmm = GaussianMixture(n_components=2, random_state=42)
gmm.fit(X)
gmm_labels = gmm.predict(X)

# Align cluster labels
def cluster_acc(true, pred):
    cost = np.zeros((2,2))
    for i in range(2):
        for j in range(2):
            cost[i,j] = np.sum((true==i)&(pred==j))
    r,c = linear_sum_assignment(-cost)
    mapping={c[i]:r[i] for i in range(2)}
    aligned=np.vectorize(mapping.get)(pred)
    return accuracy_score(true,aligned)

```

```

gmm_acc = cluster_acc(y,gmm_labels)

print("Logistic Regression Accuracy:",lr_acc)
print("GMM Accuracy:",gmm_acc)

# Decision Boundary
plt.scatter(X[:,0],X[:,1],c=y)
xx,yy=np.meshgrid(np.linspace(X[:,0].min(),X[:,0].max(),100),
                  np.linspace(X[:,1].min(),X[:,1].max(),100))
Z=lr.predict(np.c_[xx.ravel(),yy.ravel()])
plt.contour(xx,yy,Z.reshape(xx.shape))
plt.title("Logistic Regression Boundary")
plt.show()

print("""
Which learns  $P(y|x)$ ? -> Logistic Regression
Which learns  $P(x)$ ? -> GMM
Explicit likelihood? -> GMM
Why GMM generates samples? -> learns full data distribution
""")

# =====
print("\n===== Q2: Bigram Model =====\n")

text="generative ai models learn probability distributions from data."

bigram=defaultdict(Counter)
for i in range(len(text)-1):
    bigram[text[i]][text[i+1]]+=1

bigram_prob={}
for k,v in bigram.items():
    total=sum(v.values())
    bigram_prob[k]={c:v[c]/total for c in v}

def generate(temp=1.0,n=300):
    ch=random.choice(list(bigram_prob.keys()))
    out=ch
    entropy=[]
    for _ in range(n):
        chars=list(bigram_prob[ch].keys())
        probs=np.array(list(bigram_prob[ch].values()))
        probs=probs**(1/temp)

```

```

        probs/=probs.sum()
        nxt=np.random.choice(chars,p=probs)
        entropy.append(-np.sum(probs*np.log2(probs)))
        out+=nxt
        ch=nxt
    return out,np.mean(entropy)

gen_text,avg_entropy=generate()
print("Generated Text:\n",gen_text[:200])
print("Average Entropy:",avg_entropy)

def perplexity(txt):
    H,N=0,0
    for i in range(len(txt)-1):
        if txt[i] in bigram_prob:
            p=bigram_prob[txt[i]].get(txt[i+1],1e-6)
            H+=-math.log2(p);N+=1
    return 2**(H/N)

print("Perplexity:",perplexity(text))

# =====
print("\n===== Q3: Temperature Analysis =====\n")

temps=[0.3,1.0,1.8]
temp_entropy=[]
for t in temps:
    _,e=generate(t)
    temp_entropy.append(e)
    print(f"Temperature {t} -> Entropy {e}")

plt.plot(temps,temp_entropy,marker="o")
plt.title("Temperature vs Entropy")
plt.show()

# =====
print("\n===== Q4: Autoencoder Idea =====\n")

theta=np.linspace(0,2*np.pi,400)
circle=np.c_[np.cos(theta),np.sin(theta)]

pca=PCA(n_components=1)
Z=pca.fit_transform(circle)
rec=pca.inverse_transform(Z)

```

```

mse1=mean_squared_error(circle,rec)

Z_noise=Z+np.random.normal(0,0.1,Z.shape)
rec2=pca.inverse_transform(Z_noise)
mse2=mean_squared_error(circle,rec2)

print("Reconstruction MSE:",mse1)
print("With Latent Noise MSE:",mse2)

# =====
print("\n===== Q5: Lightweight GAN =====\n")

real=np.random.normal(2,1,500)
gen_mean=0
disc_loss=[]

for i in range(30):
    fake=np.random.normal(gen_mean,1,500)

    Xd=np.concatenate([real,fake]).reshape(-1,1)
    yd=np.array([1]*500+[0]*500)

    D=MLPClassifier(hidden_layer_sizes=(10,),max_iter=200)
    D.fit(Xd,yd)

    loss=1-D.score(Xd,yd)
    disc_loss.append(loss)

    gen_mean+=0.1*(real.mean()-gen_mean)

plt.hist(real,alpha=0.5,label="Real")
plt.hist(fake,alpha=0.5,label="Fake")
plt.legend()
plt.title("GAN Distribution")
plt.show()

plt.plot(disc_loss)
plt.title("Discriminator Loss")
plt.show()

# =====
print("\n===== Q6: Mode Collapse =====\n")

collapsed=np.ones(500)*real.mean()

```

```

print("Real Variance:",np.var(real))
print("Collapsed Variance:",np.var(collapsed))

plt.hist(real,alpha=0.5,label="Real")
plt.hist(collapsed,alpha=0.5,label="Collapsed")
plt.legend()
plt.show()

# =====
print("\n===== Q7: Likelihood vs GAN =====\n")

log_likelihood=gmm.score(X)
gan_loss=np.mean(disc_loss)

print("GMM Log-Likelihood:",log_likelihood)
print("GAN Avg Discriminator Loss:",gan_loss)

# =====
print("\n===== Q8: Mini Generative CLI =====\n")

choice=input("Choose Generator (1-GMM,2-Bigram,3-GAN): ")

if choice=="1":
    sample=gmm.sample(10)[0]
    print("GMM Samples:\n",sample)
    np.savetxt("gmm_output.txt",sample)

elif choice=="2":
    txt,_=generate()
    open("bigram_output.txt","w").write(txt)
    print("Bigram text saved.")

elif choice=="3":
    fake=np.random.normal(gen_mean,1,50)
    np.savetxt("gan_output.txt",fake)
    print("GAN samples saved.")

# =====
print("\n===== Q9: Metric Visualization =====\n")

gmm_metric=np.repeat(log_likelihood,len(disc_loss))
recon_metric=np.repeat(mse1,len(disc_loss))

plt.plot(gmm_metric,label="GMM Likelihood")

```

```

plt.plot(disc_loss,label="GAN Loss")
plt.plot(recon_metric,label="Reconstruction Error")
plt.legend()
plt.title("Model Metric Comparison")
plt.show()

# =====
print("\n===== Q10: Comparative Summary =====\n")

summary=[
["Model","Objective","Diversity","Stability","Likelihood?"],
["GMM","Likelihood Maximization","Medium","High","Yes"],
["GAN","Adversarial Game","High","Medium","No"],
["Autoencoder","Reconstruction","Medium","High","Implicit"]
]

for row in summary:
    print("{:<15} {:<25} {:<12} {:<12} {}".format(*row))

print("""
Summary:
Likelihood models optimize probability.
GANs optimize realism.
Autoencoders compress representations.
Different generative paradigms balance stability and diversity differently.
""")

```

